

Guía paso a paso para activar, probar y evaluar una API RESTful protegida con JWT (FastAPI)

Para completar el ejercicio práctico de evaluación de control de acceso y manipulación de tokens JWT, es necesario levantar una API funcional, probarla desde Swagger UI y luego replicar todo el flujo en Postman. Esta guía técnica describe cómo hacerlo paso a paso.

Activación del entorno local

Primero, asegúrate de tener Python 3.9 o superior instalado. Luego sigue los pasos:

1. Instala las dependencias necesarias:

```
pip install fastapi uvicorn python-jose[cryptography] passlib[bcrypt]
```

Puedes usar también `pip install -r requirements.txt` si tienes el archivo preparado.

2. Crea y ejecuta la API

Ubicado en el directorio raíz del proyecto (donde está `main.py`), ejecuta el siguiente comando:

```
uvicorn main:app --reload
```

Esto levantará el servidor en `http://127.0.0.1:8000`.

Prueba inicial desde Swagger UI

Una vez ejecutado el servidor, accede a la documentación interactiva:

URL: `http://127.0.0.1:8000/docs`

Desde aquí puedes ejecutar todos los endpoints descritos en el ejercicio. Sigue este orden:

1. Obtener un token JWT válido:

- Accede a `POST /token`
- Introduce las credenciales válidas:
 - `username: admin`

- `password: adminpass`

Haz clic en "Execute" y copia el `access_token`.

2. Autorizar con el token en Swagger

- Clic en el botón **Authorize** (en la parte superior derecha)
- Pega el token así:

Bearer TU_TOKEN_JWT

Haz clic en "Authorize" y luego "Close".

3. Prueba de endpoints protegidos

- Accede a `/api/users/me`: Debe mostrar el perfil del usuario actual
- Accede a `/api/users`: Solo debe funcionar si el rol es `admin`
- Usa `PUT /api/users/{user_id}/role` para cambiar el rol de un usuario

Puedes probar también con el usuario estándar `johndoe` y verificar que reciba un `403` al acceder a `/api/users`.

Exportar la documentación a Postman

1. Descarga el archivo OpenAPI

Desde tu navegador, entra a:

`http://127.0.0.1:8000/openapi.json`

Guárdalo como `openapi.json`.

2. Importa en Postman

- Abre Postman
- Haz clic en "Import"
- Selecciona el archivo `openapi.json`

Esto generará automáticamente la colección con todos los endpoints.

Prueba manual en Postman

1. Login (POST /token)

- Método: `POST`
- URL: `http://127.0.0.1:8000/token`
- Tipo: `x-www-form-urlencoded`
- Campos:
 - `username: admin`
 - `password: adminpass`

Guarda el `access_token` de la respuesta.

2. Probar endpoints con el token:

En cada endpoint que lo requiera:

- Ve a la pestaña **Authorization**
- Tipo: `Bearer Token`
- Token: Pega el `access_token` copiado

O bien, usa la pestaña "Headers":

Authorization: Bearer eyJhbGciOi...

3. Crear manualmente cada endpoint para simular el ejercicio

Si deseas no depender del import automático:

- Crea una colección en Postman
- Añade manualmente los siguientes endpoints:

Método	Endpoint	Requiere Token	Descripción
POST	<code>/token</code>	✗	Obtener el token JWT
GET	<code>/api/users/me</code>	✓	Ver perfil autenticado
GET	<code>/api/users</code>	✓ (solo admin)	Ver lista de todos los usuarios
PUT	<code>/api/users/{user_id}/role</code>	✓ (solo admin)	Modificar el rol de un usuario específico

En cada uno, configura el **Authorization** como Bearer y usa un **Body** adecuado (por ejemplo, en **PUT**, un JSON con `{"role": "admin"}`).

Evaluación de seguridad (como en el ejercicio práctico)

- **Simula escalamiento horizontal:** Usa un token de `johndoe` e intenta acceder a `/api/users` → debería fallar.
- **Manipula el JWT en jwt.io:** Cambia el campo `"role"` a `"admin"` y firma con `"secret"`. Prueba si el servidor lo acepta.

- **Analiza cabeceras de seguridad:** Observa la respuesta HTTP para ver si contiene:
 - `Strict-Transport-Security`
 - `Content-Security-Policy`
 - `Cache-Control`

